# Challenge Problem 3: Inverse Kinematics

Alex Martinez Zach Posch

ROBO 5000

## 1 Approach

**Problem setup:** This challenge problem was all about learning about how inverse kinematics are used in the real world to facilitate orientation and position in 3D space. In the starter code we were provided with the setup of the Panda robot and the blackboard which is floating just in front of the arm. The arm has 11 joints, ending with the end effector, although not all of them are movable. The challenge is in figuring out how to orientate the arm in the provided simulation and get it to move in a smooth path to end up with the desired drawing of CU and a Buffalo. The blackboard is centered around the point [0.5, 0.5, 0.5] in cartesian space.

**Kinematics:** There are some prebuilt functions that will be of critical use here as the math for them are quite complex. There are some public libraries like IKpy or there are some functions within Pybullet itself that can be used to go from cartesian coordinates to joint poses. There is additional information within these libraries that can also be accessed like joint type and limits as well.

**Drawing:** In order to make a drawing on the blackboard that is created in the simulation there is a provided function draw_on_blackboard() that creates a white point at wherever the end effector is in 3D space. This means that it will draw even away from the board so you have to make sure the end effector is in the correct position. In order to draw shapes you need large array of X, Y, and Z coordinates. These can be generated automatically or set manually.

## 2 Implementation Approach

### 2.1 Main Kinematics

The main difficulty with this task was just understanding the environment and how the arm moves in this new 3D space. The first thing we needed to do was to get the Panda to just make a single point on the blackboard because we can't draw complex shapes until we can do single point correctly. We learned that there were prebuilt functions to calculate the necessary joint angles of the arm so that it will be in the desired cartesian position. We found the one from Pybullet would work best.

We would put it in to move to a place and would let the simulation run for one iteration and it would never be in the place we wanted. It was then we discovered, through the help of the professor that it takes many iterations until its in the desired position. So we used Euclidean distance to calculate after each iteration how far we were from the point and once we were there we could make a dot on the board and move on to the next point.

Next was figuring out orientation because when we had it draw simple lines it would draw some but others would make the arm get stuck in a position it can't move anymore. Through trial and error we found the correct orientation and joint limits so that it can draw the desired "CU" letters on the board. We then moved the letters around on the board until it was drawn correctly.

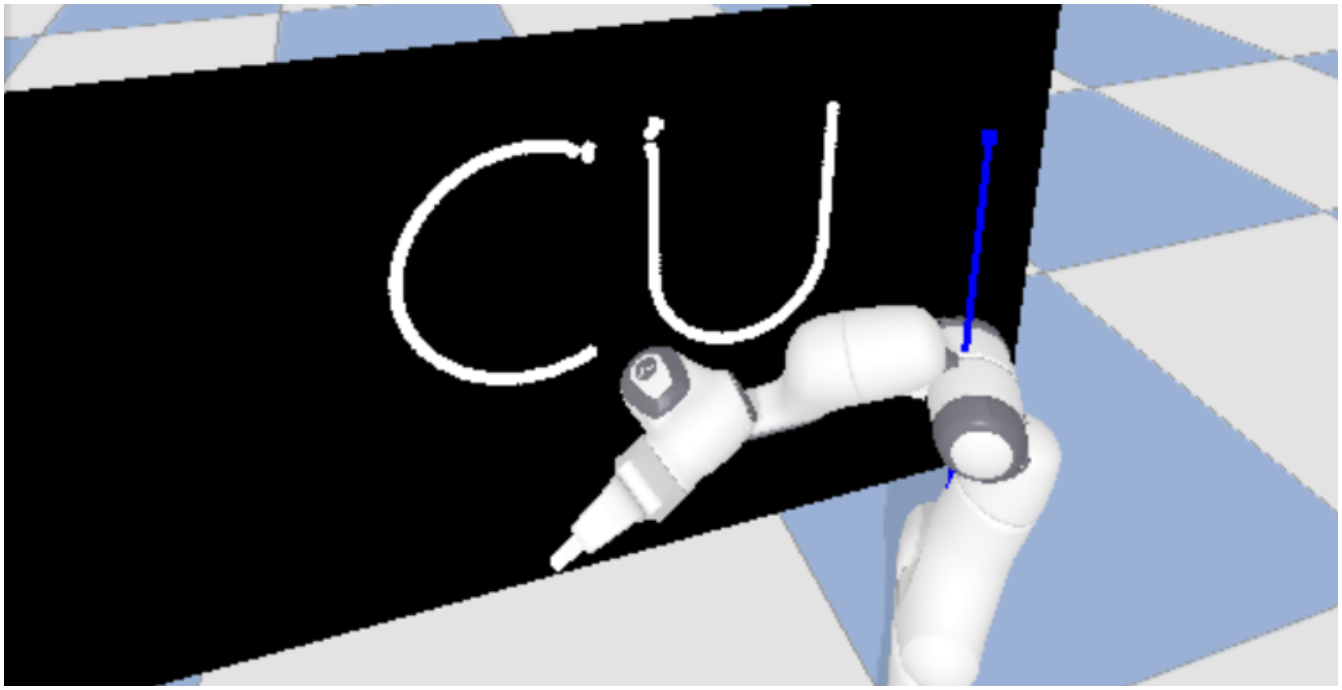


Figure 1: The end result of the Panda arm drawing the letters CU on the board. The arm then moved out of the way so we could see what was drawn.

## 2.2 Extra Credit

For this assignment we were tasked with drawing the outline of Ralphie the buffalo on the blackboard. At first this seemed like a daunting task and one that would be significantly harder than drawing two simple letters. Then we thought that after we get the points of the outline drawing the shape would follow the same basic structure as the letters.

In order to get those points we loaded in a photo of the logo into blender to be projected as a 2D image on a 3D plane, making sure that the plane is at the same cartesian position as the blackboard. Then we used blender to create an outline of the buffalo with a considerable number of points, and exported the points into a single 2D array in a function in the python code. Now we have all the points needed but found out it was too big and the arm was getting stuck again. So we used python to scale the points and move them to where we want them on the blackboard but still got stuck. After more trial and error we found that using the default settings for orientation and limits were better than the custom ones and it then drew the complete outline of the buffalo without getting stuck.

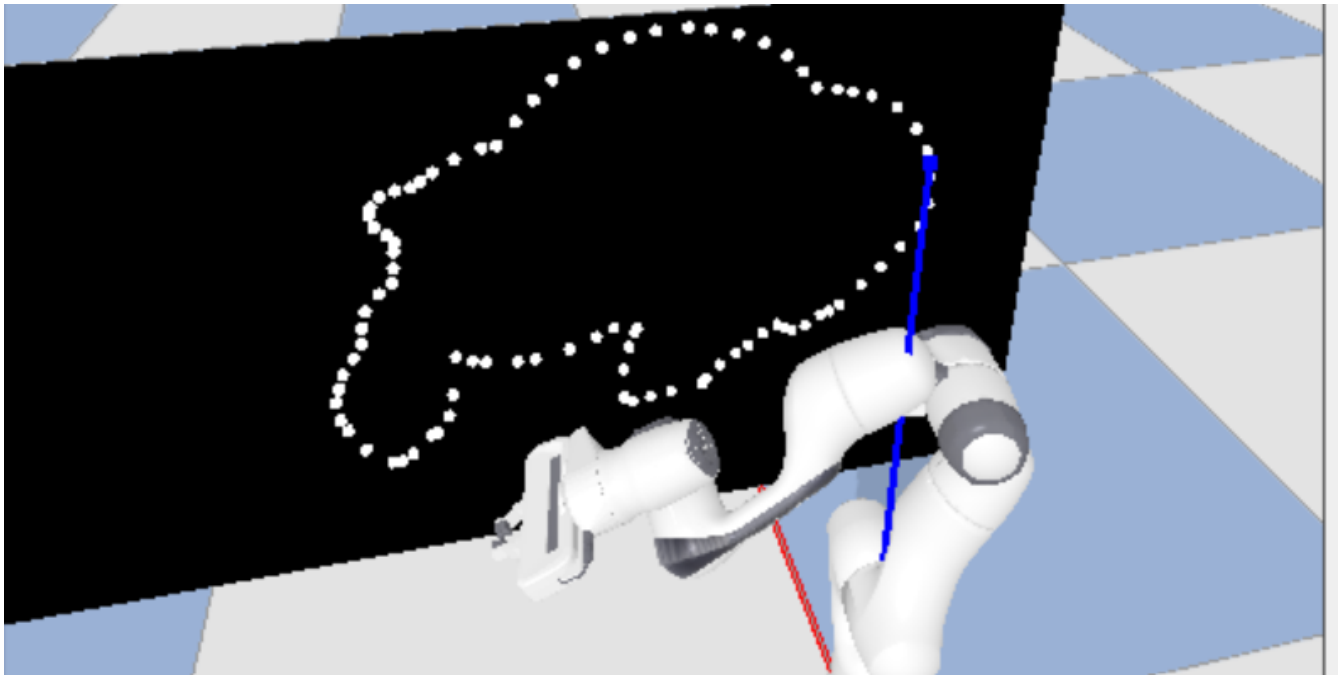


Figure 2: A second successful RRT solution with fewer sampled targets and explored branches. Compared to Figure 1, the algorithm found a more direct route to the goal, demonstrating how different random seeds can affect the search process.

## 3 Results and Discussion

After defining the kinematic constraints and point arrays, each letter and buffalo were drawn, but not without issues. Starting with an easier objective of singular points helped simplify the process until a better understanding of Pybullet was developed. Moving on from single points to arrays created new issues. First, using the default kinematic parameters proved to be insufficient for finding a full drawing path. This was likely due to singularities and invalid starting poses and angles of the arm.

To resolve this, additional parameters were added to the kinematics calculation that defined the limits of each joint, and what the starting pose should be. This maintained the end effector orientation while also finding a valid solution for the kinematics. However, it should be noted that the height the letters were drawn at had a significant impact on validity of the solution. If a letter was too low then the arm would end up in a position that either caused it to get stuck, or bend into a position that was impossible to progress through with our setup. Since each letter was called to draw separately, they each had unique kinematic solutions. This required a joint reset as a transition otherwise the starting position of the arm moving to the 2nd letter was invalid. In the future if a smoother transition was desired it may be necessary to compute the kinematics of all drawings in one pass such that the arm receives an accurate starting pose for all sequential drawings.

## 4 Conclusion

After figuring out how the environment works this challenge problem was mainly down to trial and error. First to get to the point we want in 3D space, then to draw a desired shape in a smooth continuous motion. After the main realization that we don't teleport to the cartesian position but get there after several iterations of the simulation, it was all down to figuring out orientation and arrays for the shapes we want, moving them so the arm doesn't get stuck.